

Introduction to PL/SQL

PL/SQL is a procedural extension of SQL, making it extremely simple to write procedural code that includes SQL as if it were a single language.

What is PL/SQL

PL/SQL is a block structured language. The programs of PL/SQL are logical blocks that can contain any number of nested sub-blocks. PL/SQL stands for "Procedural Language extension of SQL" that is used in Oracle. PL/SQL is integrated with Oracle database (since version 7). The functionalities of PL/SQL usually extended after each release of Oracle database. Although PL/SQL is closely integrated with SQL language, yet it adds some programming constraints that are not available in SQL.

PL/SQL Functionalities

PL/SQL includes procedural language elements like conditions and loops. It allows declaration of constants and variables, procedures and functions, types and variable of those types and triggers. It can support Array and handle exceptions (runtime errors). After the implementation of version 8 of Oracle database have included features associated with object orientation. You can create PL/SQL units like procedures, functions, packages, types and triggers, etc. which are stored in the database for reuse by applications.

With PL/SQL, you can use SQL statements to manipulate Oracle data and flow of control statements to process the data.

The PL/SQL is known for its combination of data manipulating power of SQL with data processing power of procedural languages. It inherits the robustness, security, and portability of the Oracle Database.

PL/SQL is not case sensitive so you are free to use lower case letters or upper case letters except within string and character literals. A line of PL/SQL text contains groups of characters known as lexical units. It can be classified as follows:

- Delimiters
- Identifiers
- Literals
- Comments

Structure of PL/SQL Block:

PL/SQL extends SQL by adding constructs found in procedural languages, resulting in a structural language that is more powerful than SQL. The basic unit in PL/SQL is a block. All PL/SQL Typically, each block performs a logical action in the program. A block has the following structure:

DECLARE

declaration statements;

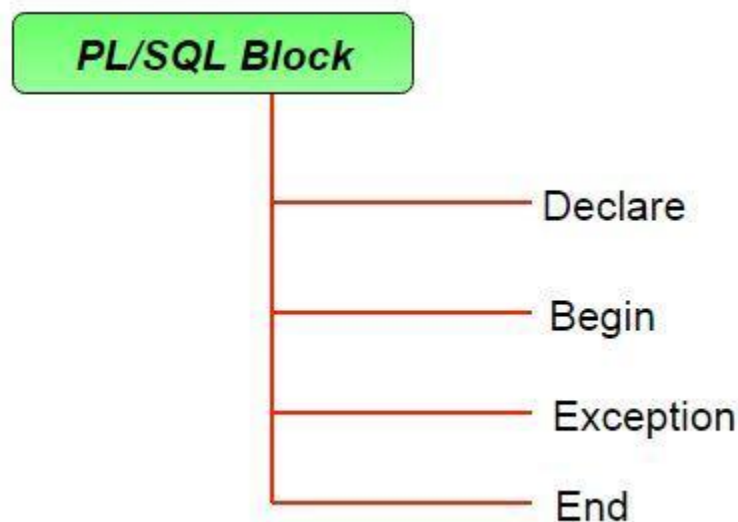
BEGIN

executable statements

EXCEPTIONS

exception handling statements

END;



- Declare section starts with **DECLARE** keyword in which variables, constants, records as cursors can be declared which stores data temporarily. It basically consists definition of PL/SQL identifiers. This part of the code is optional.
- Execution section starts with **BEGIN** and ends with **END** keyword. This is a mandatory section and here the program logic is written to perform any task like loops and conditional statements. It supports all [DML](#) commands, [DDL](#) commands and SQL*PLUS built-in functions as well.
- Exception section starts with **EXCEPTION** keyword. This section is optional which contains statements that are executed when a run-time error occurs. Any exceptions can be handled in this section.

The 'Hello World' Example

DECLARE

```
message varchar2(20):= 'Hello, World!';
```

BEGIN

```
dbms_output.put_line(message);
```

END;

/

The **end;** line signals the end of the PL/SQL block. To run the code from the SQL command line, you may need to type / at the beginning of the first blank line after the last line of the code. When the above code is executed at the SQL prompt, it produces the following result –

Hello World

PL/SQL procedure successfully completed.

The PL/SQL Identifiers

PL/SQL identifiers are constants, variables, exceptions, procedures, cursors, and reserved words. The identifiers consist of a letter optionally followed by more letters, numerals, dollar signs, underscores, and number signs and should not exceed 30 characters.

By default, **identifiers are not case-sensitive**. So you can use **integer** or **INTEGER** to represent a numeric value. You cannot use a reserved keyword as an identifier.

A delimiter is a symbol with a special meaning. Following is the list of delimiters in PL/SQL –

Delimiter	Description
+, -, *, /	Addition, subtraction/negation, multiplication, division
%	Attribute indicator
'	Character string delimiter
.	Component selector
(,)	Expression or list delimiter
:	Host variable indicator
,	Item separator
"	Quoted identifier delimiter
=	Relational operator

@	Remote access indicator
;	Statement terminator
:=	Assignment operator
=>	Association operator
	Concatenation operator
**	Exponentiation operator
<<, >>	Label delimiter (begin and end)
/*, */	Multi-line comment delimiter (begin and end)
--	Single-line comment indicator
..	Range operator
<, >, <=, >=	Relational operators
<>, '!=, ~=, ^=	Different versions of NOT EQUAL

Note: for the first time when you are executing the plsql block for getting any output on the screen. Below is the code from the sql script and the output.

set serveroutput on;

PL/SQL Program Units

A PL/SQL unit is any one of the following

■ PL/SQL block

■ Function

■ Package

■ Package body

■ Procedure

■ Trigger

■ Type

■ Type body

PL/SQL - Data Types

The PL/SQL variables, constants and parameters must have a valid data type, which specifies a storage

format, constraints, and a valid range of values.

Scalar

Single values with no internal components, such as a NUMBER, DATE, or BOOLEAN.

Large Object (LOB)

Pointers to large objects that are stored separately from other data items, such as text, graphic images, video clips, and sound waveforms.

Composite

Data items that have internal components that can be accessed individually. For example, collections and records.

Reference Pointers to other data items.

- PL/SQL Scalar Data Types and Subtypes
- PL/SQL Numeric Data Types and Subtypes
- PL/SQL Character Data Types and Subtypes
- PL/SQL Boolean Data Types
- PL/SQL Datetime and Interval Types
- PL/SQL – Variables

The name of a PL/SQL variable consists of a letter optionally followed by more letters, numerals, dollar signs, underscores, and number signs and should not exceed 30 characters. By default, variable names are not case-sensitive. You cannot use a reserved PL/SQL keyword as a variable name.

Variable Declaration in PL/SQL

PL/SQL variables must be declared in the declaration section or in a package as a global variable. When you declare a variable, PL/SQL allocates memory for the variable's value and the storage location is identified by the variable name. The syntax for declaring a variable is –

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value]

Initializing Variables in PL/SQL

Whenever you declare a variable, PL/SQL assigns it a default value of NULL. If you want to initialize a variable with a value other than the NULL value, you can do so during the declaration, using either of the following –

The **DEFAULT** keyword The assignment operator

For example –

```
counter binary_integer := 0;
```

```
greetings varchar2(20) DEFAULT 'Have a Good Day';
```

You can also specify that a variable should not have a NULL value using the NOT NULL constraint. If you use the NOT NULL constraint, you must explicitly assign an initial value for that variable.

It is a good programming practice to initialize variables properly otherwise, sometimes programs would produce unexpected results. Try the following example which makes use of various types of variables –

DECLARE

```
a integer := 10;
```

```
b integer := 20;
```

```
c integer;
```

```
f real;
```

BEGIN

```
c := a + b;
```

```
dbms_output.put_line('Value of c: ' || c);
```

```
f := 70.0/3.0;
```

```
dbms_output.put_line('Value of f: ' || f);
```

END;

```
/
```

Variable Scope in PL/SQL

PL/SQL allows the nesting of blocks, i.e., each program block may contain another inner block. If a variable is declared within an inner block, it is not accessible to the outer block. However, if a variable is declared and accessible to an outer block, it is also accessible to all nested inner blocks. There are two types of variable scope

■ **Local variables** – Variables declared in an inner block and not accessible to outer blocks.

 **Global variables** – Variables declared in the outermost block or a package.

Following example shows the usage of Local and Global variables in its simple form.

DECLARE

-- Global variables

num1 number := 95;

num2 number := 85;

BEGIN

dbms_output.put_line('Outer Variable num1: ' || num1);

dbms_output.put_line('Outer Variable num2: ' || num2);

DECLARE

-- Local variables

num1 number := 195;

num2 number := 185;

BEGIN

dbms_output.put_line('Inner Variable num1: ' || num1);

dbms_output.put_line('Inner Variable num2: ' || num2);

END;

END;

/

PL/SQL - Constants and Literals

A constant holds a value that once declared, does not change in the program. A constant declaration specifies its name, data type, and value, and allocates storage for it. The declaration can also impose the NOT NULL constraint.

Declaring a Constant

A constant is declared using the **CONSTANT** keyword. It requires an initial value and does not allow that value to be changed. For example –

```
PI CONSTANT NUMBER := 3.141592654;
```

DECLARE

```
-- constant declaration
```

```
pi constant number := 3.141592654;
```

```
-- other declarations
```

```
radius number(5,2);
```

```
dia number(5,2);
```

```
circumference number(7, 2);
```

```
area number (10, 2);
```

BEGIN

```
-- processing
```

```
radius := 9.5;
```

```
dia := radius * 2;
```

```
circumference := 2.0 * pi * radius;
```

```
area := pi * radius * radius;
```

```
-- output
```

```
dbms_output.put_line('Radius: ' || radius);
```

```
dbms_output.put_line('Diameter: ' || dia);
```

```
dbms_output.put_line('Circumference: ' || circumference);
```

```
dbms_output.put_line('Area: ' || area);
```

```
END;
```

```
/
```

The PL/SQL Literals

A literal is an explicit numeric, character, string, or Boolean value not represented by an identifier. For example, TRUE, 786, NULL, 'tutorialspoint' are all literals of type Boolean, number, or string. PL/SQL, literals are case-sensitive. PL/SQL supports the following kinds of literals –

■ **Numeric Literals**

■ **Character Literals**

■ **String Literals**

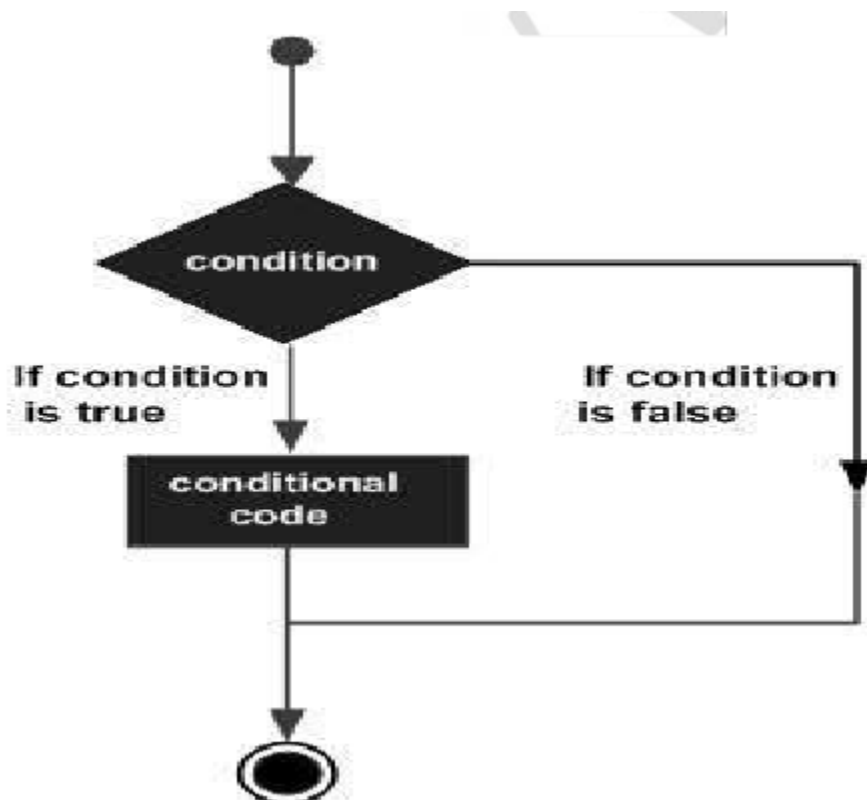
■ **BOOLEAN Literals**

■ **Date and Time Literals**

PL/SQL - Conditions

Decision-making structures require that the programmer specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

Following is the general form of a typical conditional (i.e., decision making) structure found in most of the programming languages –



PL/SQL programming language provides following types of decision-making statements. Click the following links to check their detail.

S.No

Statement & Description

IF - THEN statement

- 1 The **IF statement** associates a condition with a sequence of statements enclosed by the keywords **THEN** and **END IF**. If the condition is true, the statements get executed and if the condition is false or NULL then the IF statement does nothing.

IF-THEN-ELSE statement

- 2 **IF statement** adds the keyword **ELSE** followed by an alternative sequence of statement. If the condition is false or NULL, then only the alternative sequence of statements get executed. It ensures that either of the sequence of statements is executed.

IF-THEN-ELSIF statement

- 3 It allows you to choose between several alternatives.

Case statement

- 4 Like the IF statement, the **CASE statement** selects one sequence of statements to execute. However, to select the sequence, the CASE statement uses a selector rather than multiple Boolean expressions. A selector is an expression whose value is used to select one of several alternatives.

nested IF-THEN-ELSE

- 5 You can use one **IF-THEN** or **IF-THEN-ELSIF** statement inside another **IF-THEN** or **IF-THEN-ELSIF** statement(s).